

Lab0: Gitlab
顾心怡 25300120166

一、

1、你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

并没有协作开发代码的经历。但是有写作撰写文档 docx 的经历。一般会开一个共享文档同步进度，但共享文档由于是实时跟新，想要找回之前修改的某个版本比较困难。

2、Git 为什么要设计“暂存-提交”两个步骤？

Git 的暂存区使得开发修改在进入正式版本记录之前，能有一个确认区域，保证提交的内容是经过确认的，这能确保正式版本历史的记录清晰简明，便于后续的查看。此外，暂存区支持部分提交，既能提交部分已经修改好的代码，也能暂存其余未完成的修改。

3、git branch 和 git branch -a 的区别是什么？查阅资料并回答。

git branch 和 git branch -a 都是用于查看 git 分支的命令，但查看的范围不同。

1、git branch 会列出本地仓库中所有的本地分支，可用于查看本地有哪些分支，也能用于确认当前所在分支。

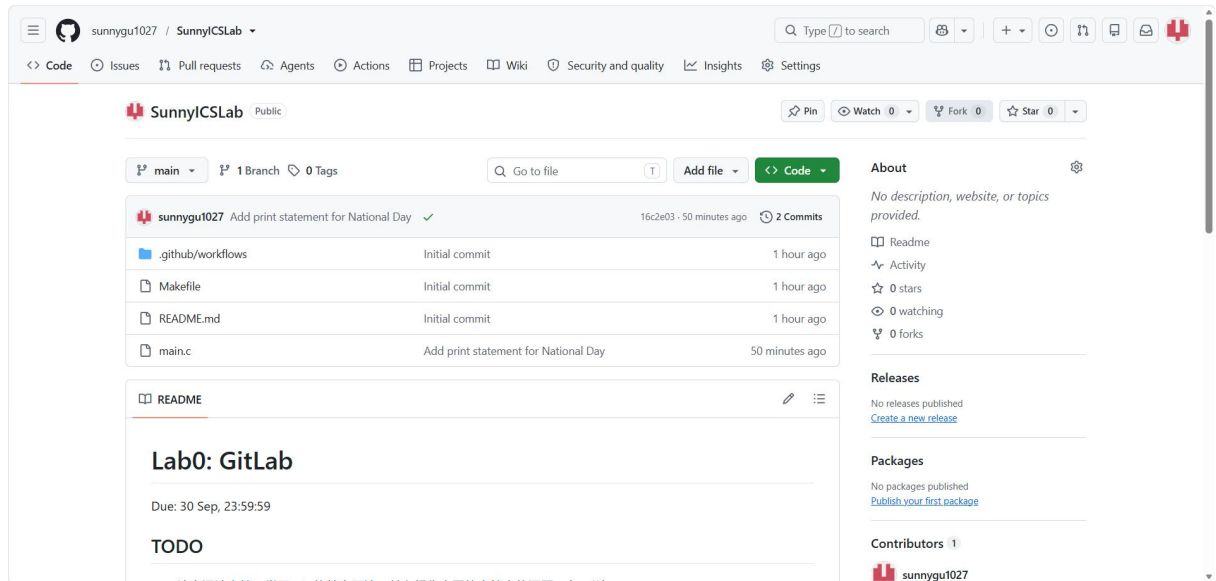
2. git branch -a 能列出本地分支和地仓库中记录的远程跟踪分支的所有分支。

远程分支是本地对远程仓库分支的缓存，通常以 remotes/或 origin/为前缀（例如 remotes/origin/main）。通过 git fetch 或 git pull 更新，能看到最新的远程分支列表。

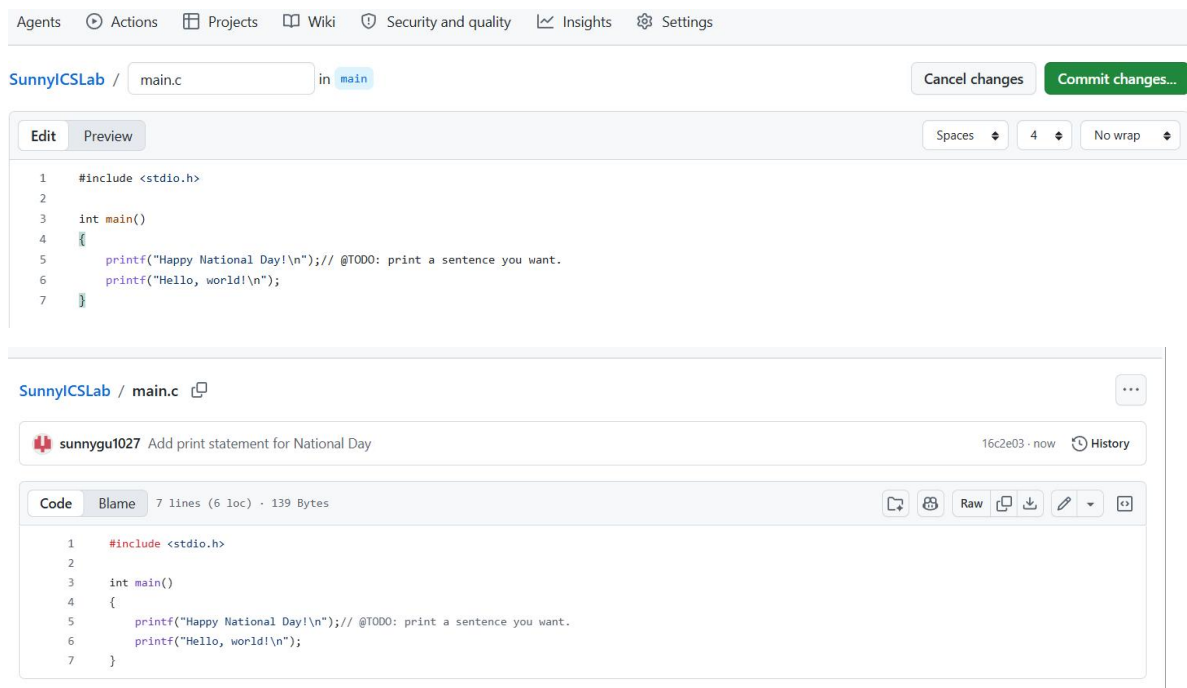
当需要查看远程仓库有哪些分支，或者需要确认本地分支与远程分支的对应关系（例如确认本地分支是否跟踪了远程分支）时，可以使用 git branch -a，但不能直接在远程分支上写代码。

二、使用此仓库建立个人仓库，完成 main.c 文件中的 TODO 部分并进行一次 commit。（50 分）

在注册好 github 邮箱并配置 SSH 后，在模板仓库点击 Use this template 并选择 Create a new repository，建立个人仓库。



点击 main.c 文件并点击编辑功能，加入修改内容后，commit change.



三、在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15）

1、Commit Message 规范：在写 commit message 时，若遵循像 Angular 规范等格式化写法，能使得版本历史记录更加清晰，方便迅速浏览修改过程，可以过滤某些 commit（比如文档改动），且可以直接从 commit 生成 Change log。其具体格式为：

```
<type><scope>: <subject>
// 空一行
<body>
// 空一行
<footer>
```

通过下载 Commitizen，能生成符合格式的 commit message，用 validate-commit-msg 检查项目的 Commit message 是否符合格式。

2、Git Flow 分支控制：介绍了 Gitflow 分支管理工作流程，其中关键分支包括 master、develop（开发过程中操作分支）、feature（新功能分支）、release（测试分支）、hotfix（线上紧急修复）。同时，梳理了从新功能开发、测试验证到正式发布及线上修复的代码流转流程，并提供了手动操作与 Gitflow 扩展工具两种实践方式。

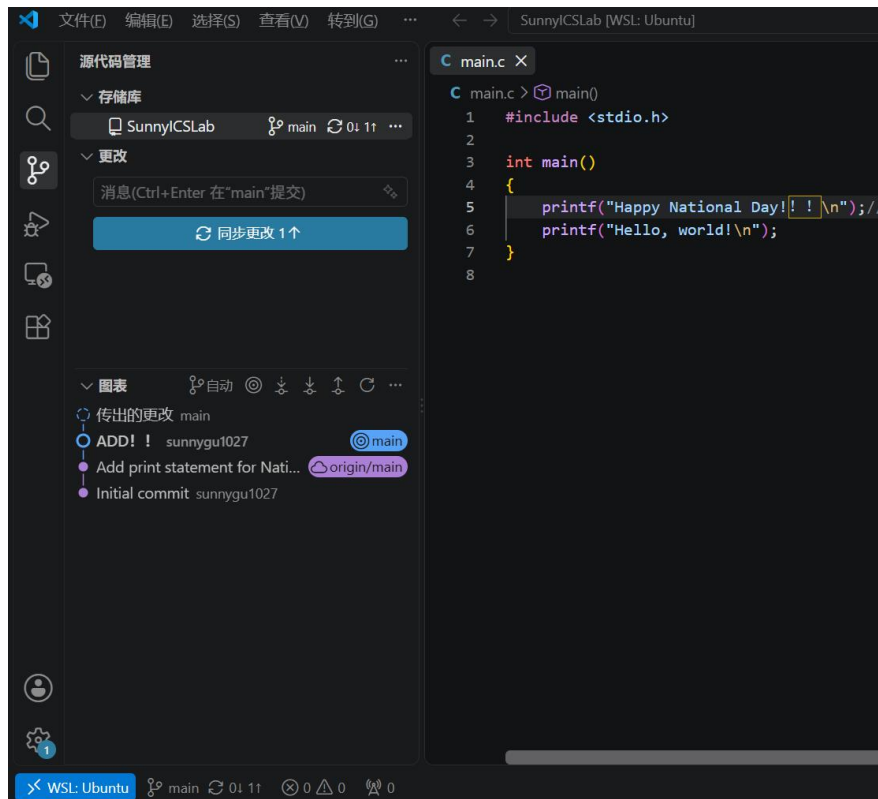
学习 git，能在后续调试、修改代码时有清晰的版本历史记录，避免想找回之前版本时丢失记录。git 分支则能在不影响主要版本的情况下调试某个模块的功能与修改。Git 也便于在多人协作开发并行开发时，同步合并修改，解决合并冲突。

四、学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交(10 分)。随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature），并处理发生的合并冲突(10 分)。

先用 git clone 克隆远程仓库到本地

```
sunny@sunny:/mnt/c/Users/34372$ git clone git@github.com:sunnygu1027/SunnyICSLab.git
Cloning into 'SunnyICSLab'...
Enter passphrase for key '/home/sunny/.ssh/id_ed25519':
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 2), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), 4.46 KiB | 325.00 KiB/s, done.
Resolving deltas: 100% (2/2), done.
sunny@sunny:/mnt/c/Users/34372$ git pull
fatal: not a git repository (or any parent up to mount point /mnt)
Stopping at filesystem boundary (GIT_DISCOVERY_ACROSS_FILESYSTEM not set).
sunny@sunny:/mnt/c/Users/34372$ cd ~/SunnyICSLab
-bash: cd: /home/sunny/SunnyICSLab: No such file or directory
sunny@sunny:/mnt/c/Users/34372$ cd ~/SunnyICSLab ls
-bash: cd: too many arguments
sunny@sunny:/mnt/c/Users/34372$ mv /mnt/c/Users/34372/SunnyICSLab ~/
sunny@sunny:/mnt/c/Users/34372$ cd ~/SunnyICSLab
ls
Makefile README.md main.c
sunny@sunny:~/SunnyICSLab$ |
```

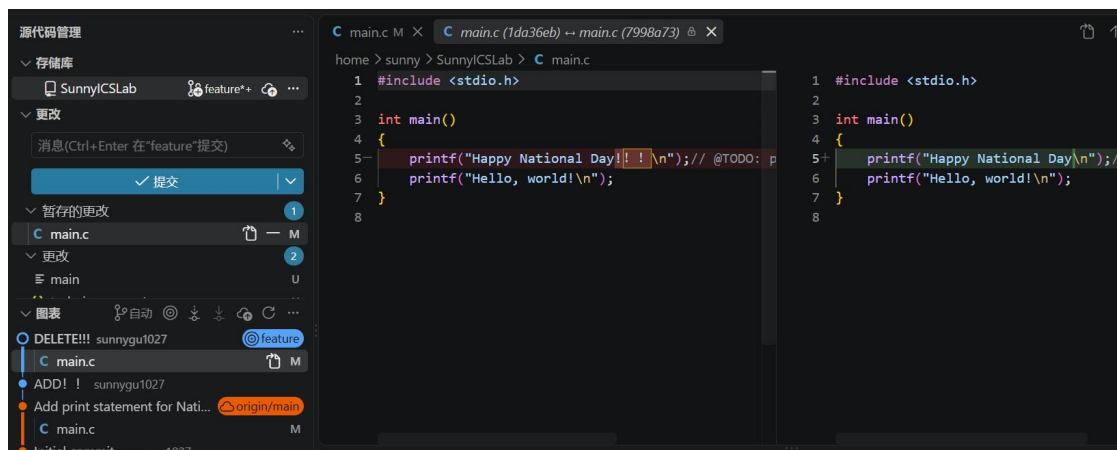
在 vscode 中打开对应仓库文件夹，并在 main 分支上对 main.c 进行一次 ADD!! 的修改并提交。



新建 feature 分支并 git switch feature 跳转到 feature 分支

```
sunny@sunny:~/SunnyICSLab$ git branch feature
sunny@sunny:~/SunnyICSLab$ git switch feature
Switched to branch 'feature'
```

在 feature 分支上对 main.c 进行一次 DELETE!!! 的修改并提交。



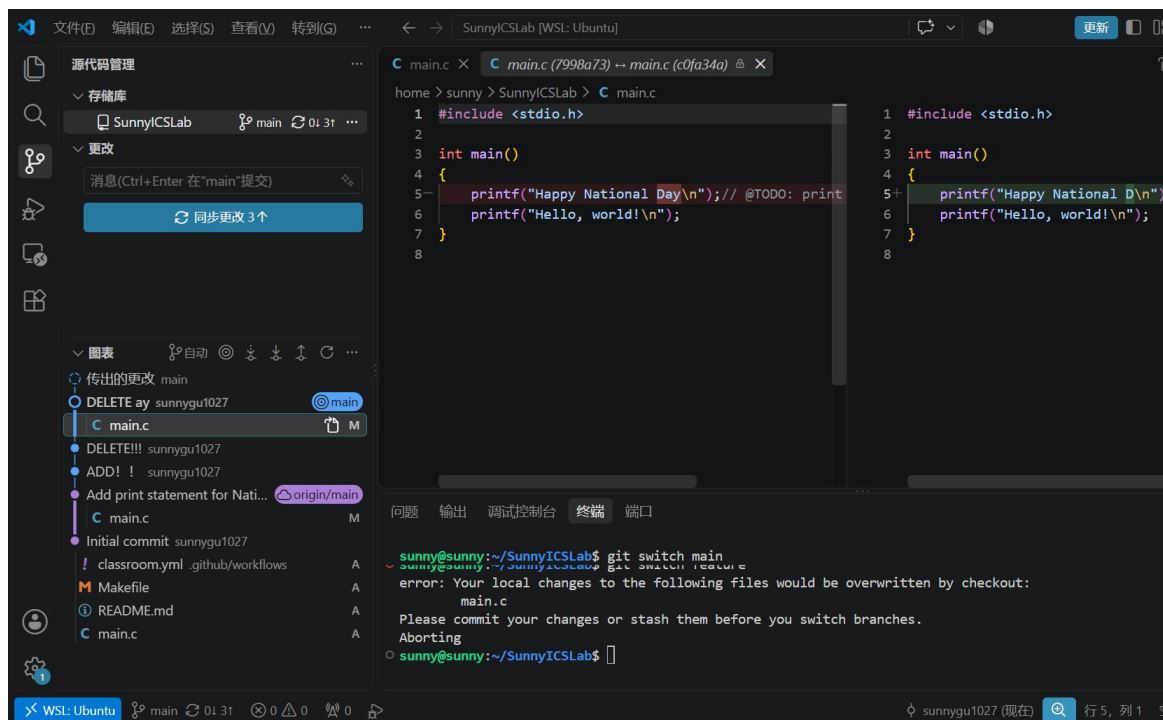
切换到 main 分支，将 feature 分支 merge 合并到 main 分支，未产生冲突，原因可能是并未在同一文件的同一地方进行不同的修改

```

sunny@sunny:~/SunnyICSLab$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
sunny@sunny:~/SunnyICSLab$ git merge feature
Updating 1da36eb..7998a73
Fast-forward
 main.c | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

```

于是在 main 分支上进行 delete ay 的修改。



再次将 feature 分支 merge 合并到 main 分支。同一个地方在两个分支上修改的内容不同，出现合并冲突。

```
C main.c 3,! X C main.c (7998a73) ↔ main.c (c0fa34a) X
C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6      <<<<<< HEAD (当前更改)
7      printf("Happy National D\n");// @TODO: print a sentence you want.
8      =====
9      printf("Happy National!!!\n");// @TODO: print a sentence you want.
10     >>>>>> feature (传入的更改)
11     printf("Hello, world!\n");
12 }

问题 3 输出 调试控制台 终端 端口

sunny@sunny:~/SunnyICSLab$ git switch feature
Auto-Merging
sunny@sunny:~/SunnyICSLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

手动选择采用传入的修改。保存上传提交。成功处理合并冲突，将 feature 分支合并到 main 分支。

